

PATRONES GRASP

General Responsibility Assignment Software Patterns

¿A quién le toca hacer qué en el código?

Creator

Info Expert

Low Coupling (Bajo Acoplamiento)

High Cohesion (Alta cohesión)

Controller

Polymorphism

Pure Fabrication

Indirection

Prot. Variations

¿Qué son los patrones GRASP?



Principios para decidir CÓMO asignar responsabilidades a las clases y objetos en diseño orientado a objetos.

— Craig Larman, Applying UML and Patterns



No son código

Son criterios de buen juicio para DISEÑAR antes de programar



1 pregunta clave

¿Qué clase es la más adecuada para esta responsabilidad?



9 patrones

Cada uno responde un aspecto diferente del diseño OO

¿Por qué importa asignar bien las responsabilidades?

✗ Mal diseñado

1. Clases enormes que hacen de todo (God Class)
2. Cambiar algo rompe otras 5 clases
3. Código imposible de reutilizar en otro proyecto
4. Nadie entiende qué hace cada clase
5. Probar el sistema es una pesadilla

✓ Bien diseñado (GRASP)

1. Clases pequeñas, enfocadas y reutilizables
2. Cambios localizados sin efectos secundarios
3. Código que se puede probar fácilmente
4. Cualquier desarrollador entiende el diseño
5. Escalar el sistema no duele

El código típico del primer proyecto

Sistema de una tienda — todo en un solo script, tal como lo aprenden primero:

```
●●●
# tienda.py - proyecto de primer semestre
productos = []
ventas = []

def agregar_producto(nombre, precio, stock):
    productos.append({'nombre': nombre, 'precio': precio, 'stock': stock})

def vender(nombre_producto, cantidad):
    for p in productos:
        if p['nombre'] == nombre_producto:
            if p['stock'] >= cantidad:
                p['stock'] -= cantidad
                total = p['precio'] * cantidad
                ventas.append({'producto': nombre_producto,
                              'cantidad': cantidad, 'total': total})
                # enviar email de confirmacion
                print(f'Email enviado: compraste {cantidad} x {nombre_producto}')
                # guardar en archivo txt
                with open('ventas.txt', 'a') as f:
                    f.write(f'{nombre_producto},{cantidad},{total}\n')
                # calcular comision del vendedor
                comision = total * 0.05
                print(f'Comision: ${comision:.2f}')
                return total
            else:
                print('Stock insuficiente')
                return 0
    print('Producto no encontrado')
    return 0
```



Funciona... pero ¿qué pasa cuando el sistema crece?

El mismo script... 6 meses después



Una función de 200 líneas

vender() ahora maneja pagos, emails, inventario, impuestos, cupones y reportes. Nadie se atreve a tocarla.



Todo acoplado a todo

Cambiar el formato del email rompe el cálculo de impuestos. Cambiar la BD rompe el cálculo de comisiones.



Imposible de probar

Para probar que el descuento es correcto, necesitas crear productos, ventas, archivos y conexión a BD.



Nadie puede trabajar en paralelo

Juan y María modifican vender() al mismo tiempo. Conflictos en git. El código de uno siempre rompe al otro.



No se puede reutilizar nada

El jefe pide una app móvil. Todo el código asume archivos .txt y prints. Hay que reescribir todo desde cero.

Los 9 patrones GRASP



Creator

¿Quién crea objetos?



Info Expert

¿Quién tiene los datos?



Low Coupling

Minimizar dependencias



High Cohesion

1 clase, 1 responsabilidad



Controller

Coordinar sin hacer



Polymorphism

Sin if/switch interminables



Pure Fabrication

Inventar clases necesarias



Indirection

Intermediarios que desacoplan



Prot. Variations

Proteger de los cambios



BLOQUE 1

Patrones Fundamentales

Creator · Information Expert · Low Coupling · High Cohesion

Creator — ¿Quién crea los objetos?

Analogía

*Un arquitecto no construye la casa él solo. Encarga al contratista que tiene materiales y planos.
En código: quien tiene la información y recursos para crear un objeto, ese es quien debe crearlo.*

Asigna a B la responsabilidad de crear A si:



B contiene o agrega objetos de tipo A (B es una colección de A's)



B registra o lleva historial de objetos A



B usa directamente objetos de tipo A (dependencia fuerte)



B tiene los datos de inicialización que A necesita para crearse

Creator en acción: Pedido y LineaDePedido

¿Pedido crea LineaDePedido? Sí — Pedido CONTIENE una lista de líneas y TIENE los datos para crearlas.

✗ Mal — UI crea objetos internos

```
# ✗ La UI crea objetos del dominio

class PantallaPedido:
    def agregar_producto(self, prod, cant):
        # La UI conoce los internos de Pedido
        linea = LineaDePedido(prod, cant)
        pedido.lineas.append(linea)
        # Problema: UI acoplada a detalles
```

✓ Bien — Pedido crea sus líneas

```
# ✓ Pedido crea sus propias líneas

class Pedido:
    def __init__(self):
        self.lineas = []

    # Pedido contiene a LineaDePedido
    # → Pedido es el Creator
    def agregar(self, producto, cantidad):
        self.lineas.append(
            LineaDePedido(producto, cantidad)
        )
```



Regla simple: Si A está DENTRO de B, o B USA a A frecuentemente → B crea A



Information Expert — Dale la tarea a quien tiene los datos



Analogía: el detective

*Si un detective quiere saber quién cometió el crimen, pregunta a quien estuvo en el lugar, no a alguien de otra ciudad.
En código: la clase que tiene los DATOS es quien debe calcular o hacer la operación con esos datos.*

Principio: Asigna la responsabilidad a la clase que ya tiene la información necesaria para cumplirla.

¿Quién calcula el total del pedido?

LineaDePedido

Tiene: precio y cantidad

→ **Calcula: subtotal propio**



Pedido

Tiene: lista de líneas

→ **Calcula: total del pedido**

Cada clase calcula lo que le corresponde según sus datos

✗ Mal — clase ajena hace el cálculo

```
# ✗ Clase ajena calcula el total
class ReportePedidos:
    def total(self, pedido):
        t = 0
        # Accede a internos de otras clases
        for l in pedido.lineas:
            t += l.precio * l.cantidad
        return t # NO es el experto
```

✓ Bien — cada clase calcula lo suyo

```
# ✓ Pedido tiene sus líneas → es el experto
class Pedido:
    def get_total(self):
        return sum(l.get_subtotal()
                    for l in self.lineas)

# LineaDePedido es experta de su subtotal
class LineaDePedido:
    def get_subtotal(self):
        return self.producto.precio * self.cantidad
```

Low Coupling — Minimiza las dependencias entre clases



Analogía: músicos de una banda

Un guitarrista que SOLO puede tocar con UN baterista en el mundo entero tiene un acoplamiento terrible. Si ese baterista se enferma, no hay concierto. Un músico que puede tocar con cualquier baterista → bajo acoplamiento.

El acoplamiento como un 'termómetro de dependencias':

Bajo

Acoplamiento de datos
(via métodos)

Medio

Acoplamiento de control
(flags que cambian
comportamiento)

Alto

Acoplamiento global
(variables compartidas)

Crítico

Acoplamiento de contenido
(acceso a privados de otra clase)

Low Coupling — Minimiza las dependencias entre clases

✗ Acoplamiento a implementación

```
# ✗ Alto acoplamiento
class Tienda:
    def __init__(self):
        # Acoplada a implementación concreta
        self.visa = VisaGateway()

    def pagar(self, monto):
        self.visa.procesar_visita(monto)
        # Cambiar a Mastercard → reescribir Tienda
```

✓ Acoplamiento a abstracción

```
# ✓ Bajo acoplamiento
class Tienda:
    def __init__(self, pasarela):
        # Depende de la ABSTRACCIÓN
        self.pasarela = pasarela # inyección

    def pagar(self, monto):
        self.pasarela.procesar(monto) # flexible
        # Cambiar pasarela → solo cambiar arg
```



High Cohesion — Una clase, una responsabilidad



Analogía: restaurante especializado

Un restaurante de sushi no repara coches ni corta cabello. Alta cohesión = hace solo cosas relacionadas entre sí. Una clase cohesionada solo tiene métodos que trabajan con sus propios datos.

✗ God Class — hace todo

```
# ✗ Baja cohesión — God Class
class GestorSistema:
    def guardar_usuario(self, ...): ...
    def enviar_email(self, ...): ...
    def calcular_impuesto(self, ...): ...
    def generar_pdf(self, ...): ...
    def procesar_pago(self, ...): ...
    def autenticar_usuario(self, ...): ...
    # ...50 métodos más...
```

✓ Clases pequeñas y enfocadas

```
# ✓ Alta cohesión — clases enfocadas
class RepositorioUsuario: ...
# → solo guarda/lee usuarios

class ServicioEmail: ...
# → solo envía emails

class CalculadorImpuesto: ...
# → solo calcula impuestos

class GeneradorPDF: ...
# → solo genera PDFs
```



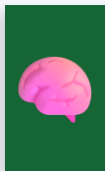
Resumen: los 4 patrones fundamentales



Creator

Regla: Si B contiene/usa/registra A $\rightarrow$ B crea A

✓ Evita creación dispersa y acoplamiento innecesario



Info Expert

Regla: ¿Quién tiene los datos? Esa clase tiene la responsabilidad

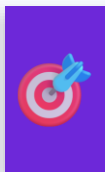
✓ Lógica donde viven los datos. Cohesión natural



Low Coupling

Regla: Depende de abstracciones, no de implementaciones

✓ Cambiar A no rompe B, C, D. Usa interfaces



High Cohesion

Regla: Una clase = una responsabilidad bien definida

✓ Clases pequeñas, legibles, testeables. Sin God Classes

BLOQUE 2

Patrones Estructurales

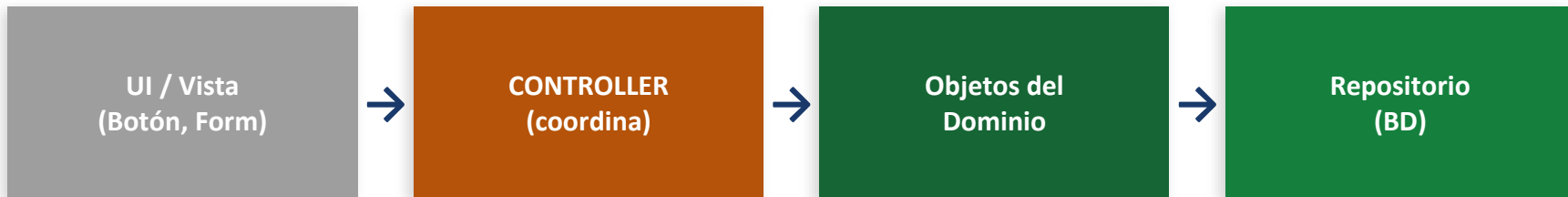
Controller · Polymorphism · Pure Fabrication · Indirection · Protected Variations

Controller — El coordinador que no ejecuta



Analogía: director de orquesta

El director no toca ningún instrumento: coordina para que la sinfonía suene bien. El Controller no ejecuta la lógica del negocio, solo delega a quienes sí la tienen. Es el primer objeto del dominio que recibe un evento del sistema.



Anti-patrón: Controller inflado

Un Controller que ejecuta TODA la lógica él solo se convierte en una God Class disfrazada.

Regla: si tu Controller tiene 500 líneas → algo está mal. DEBE delegar.



Controller correcto

- Recibe el evento del sistema
- Valida quién debe hacer qué
- Llama a objetos del dominio
- Nunca tiene lógica de negocio propia
- Es delgado: 50-100 líneas máximo

Controller separa la UI de la lógica de negocio

✗ Lógica de negocio en la UI

```
# ✗ Botón tiene lógica de negocio
def on_click_vender(event):
    producto = buscar_producto(id)
    if producto.stock >= cantidad:
        producto.reducir_stock(cantidad)
        venta = Venta(producto, cantidad)
        db.guardar(venta)
        calcular_comision(venta)
    # UI conoce todo el dominio
```

✓ Controller delegado que delega

```
# ✓ UI solo llama al Controller
def on_click_vender(event):
    ctrl.registrar_venta(id_prod, cant)

# Controller coordina y delega
class VentaController:
    def registrar_venta(self, id_prod, cant):
        producto = self.catalogo.buscar(id_prod)
        producto.reducir_stock(cant)
        venta = Venta(producto, cant)
        self.registro_ventas.agregar(venta)
```

Polymorphism — Adiós a los switch interminables



Analogía: enchufes USB-C

Un cargador USB-C funciona con cualquier teléfono que tenga ese puerto, sin importar la marca. Todos implementan el mismo contrato. En código: si distintos tipos de objetos tienen el mismo comportamiento, define una interfaz común.

✗ switch que crece con cada nuevo tipo

```
# ✗ if/elif que crece con cada tipo
def procesar(pago):
    if pago.tipo == "EFECTIVO":
        # lógica efectivo...
        pass
    elif pago.tipo == "VISA":
        # lógica Visa...
        pass
    elif pago.tipo == "CRIPTO":
        # lógica cripto...
        pass
    # Añadir PayPal → modificar aquí
```

✓ Nueva clase sin modificar el resto

```
# ✓ Cada tipo implementa su propio método
from abc import ABC, abstractmethod

class Pago(ABC):
    @abstractmethod
    def procesar(self): ...

class PagoEfectivo(Pago):
    def procesar(self): ...

class PagoVisa(Pago):
    def procesar(self): ...

# Añadir PayPal → nueva clase, sin tocar nada
class PagoPayPal(Pago):
    def procesar(self): ...
```

Pure Fabrication — Inventa la clase que necesitas



Analogía: departamento de logística

El departamento de logística no fabrica ni vende. Es una estructura artificial para un propósito específico. No existe en el 'dominio' del negocio pero mejora enormemente el sistema. Así son las clases Pure Fabrication.

¿Dónde pongo el acceso a la base de datos? → En una clase artificial: el Repositorio

```

●●●
# La clase del dominio (limpia, sin infraestructura)
class Pedido:
    def __init__(self): self.lineas = []
    def get_total(self): ... # Solo lógica de negocio

# ✅ Pure Fabrication: clase INVENTADA para persistencia
class PedidoRepositorio:
    def guardar(self, pedido): ...
    def buscar_por_id(self, id_pedido): ...
    # INSERT/SELECT aquí, Pedido no sabe de BD

# ✅ Pure Fabrication: clase INVENTADA para notificaciones
class NotificadorPedido:
    def notificar_confirmacion(self, pedido): ...
    # lógica de email aquí, Pedido no sabe de emails
    
```

 Repository, Service, DAO, DTO son todos Pure Fabrications. Son la base de la arquitectura en capas.

Indirection — El intermediario que desacopla



Analogía: el banco como intermediario

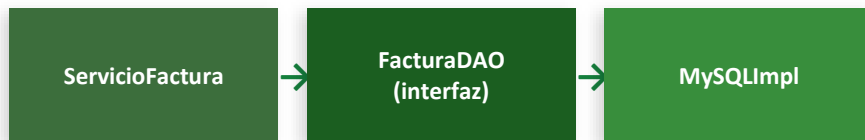
Quando pagas con tarjeta, no hay acuerdo directo entre la tienda y tu banco. VISA es el intermediario. Si tu banco cambia su sistema interno, la tienda no se entera. Eso es Indirection: intermediario que absorbe el cambio.

Sin intermediario (acoplado):



✗ Directo — acoplado a MySQL

Con intermediario (desacoplado):



✓ Via interfaz — desacoplado de MySQL

Tipos de intermediarios en arquitectura:

DAO / Repository

Lógica ↔ Base de datos

Desacopla
persistencia

Adapter

Clase ↔ Clase
incompatible

Traduce entre
interfaces

Facade

Cliente ↔ Subsistema
complejo

Simplifica la interfaz

Service Layer

UI ↔ Dominio

Coordina
operaciones

Protected Variations — Protege tu código de los cambios



Analogía: adaptador de viaje universal

Un viajero lleva un adaptador universal porque no sabe si el próximo país tendrá enchufes tipo A, B o F. El adaptador lo protege de esa variación futura. En código: identifica los puntos que cambiarán e instala una interfaz estable alrededor de ellos.

Puntos de variación comunes que debes proteger:



Métodos de pago

Hoy Visa. Mañana crypto, OXXO, PayPal



Base de datos

Hoy MySQL. Mañana PostgreSQL, MongoDB



Notificaciones

Hoy email. Mañana SMS, push, Slack



Formatos exportación

Hoy PDF. Mañana Excel, JSON, XML



Proveedor cloud

Hoy AWS. Mañana Azure, GCP



APIs externas

Versiones que cambian sin aviso

```
# ✅ Interfaz estable que te protege de la variación
from abc import ABC, abstractmethod
class ServicioNotificacion(ABC):
    @abstractmethod
    def notificar(self, destino: str, msg: str): ...
class NotificacionEmail(ServicioNotificacion): ...
class NotificacionSMS(ServicioNotificacion): ...
# Añadir Slack: nueva clase. ServicioUsuario NO cambia.
```

Los patrones no trabajan solos



↔ Indirection — conecta todos los anteriores creando objetos intermediarios

La cadena GRASP:

Info Expert identifica al responsable → si viola cohesión, usa Pure Fabrication → para desacoplar usa Indirection → para variaciones usa Polymorphism → para proteger el futuro usa Protected Variations.

Los 9 patrones de un vistazo

	Creator	¿Quién crea A?	Evita: Creación dispersa
	Info Expert	¿Quién tiene los datos?	Evita: Lógica en lugar equivocado
	Low Coupling	¿Cómo reducir dependencias?	Evita: Efecto dominó al cambiar
	High Cohesion	¿Cómo evitar God Class?	Evita: Clases que hacen de todo
	Controller	¿Quién recibe el evento?	Evita: Lógica de negocio en la UI
	Polymorphism	¿Cómo manejar variaciones?	Evita: if/switch que no para de crecer
	Pure Fabrication	¿Y si ninguna clase encaja?	Evita: Contaminar clases del dominio
	Indirection	¿Cómo desacoplar A de B?	Evita: Acoplamiento a implementaciones
	Prot. Variations	¿Cómo protegerme del cambio?	Evita: Código frágil ante variaciones

¿Cuándo aplico cada patrón?

Si... Necesito crear un objeto. ¿A quién le pido?	 Creator
Si... ¿Qué clase debe ser responsable de este cálculo?	 Info Expert
Si... Mi clase depende de demasiadas otras	 Low Coupling
Si... Mi clase crece sin parar y ya no entiendo qué hace	 High Cohesion
Si... La UI tiene lógica de negocio. ¿Cómo la separo?	 Controller
Si... Tengo un if/switch que crece con cada tipo nuevo	 Polymorphism
Si... Necesito persistencia pero no quiero contaminar mi dominio	 Pure Fabrication
Si... Dos clases que no deberían conocerse necesitan colaborar	 Indirection
Si... Esta parte va a cambiar en el futuro. ¿Cómo me protejo?	 Prot. Variations

Ejercicio 1: La UI calcula el descuento



Escenario:

La clase ClienteUI tiene un método calcularDescuento() que accede a los atributos privados de Pedido para sumar los totales y aplicar reglas de negocio.



Pista:

¿Quién tiene la información (líneas del pedido, reglas de negocio) necesaria para calcular el descuento?



Respuesta:

Information Expert — Pedido tiene sus propias líneas y reglas. El cálculo del descuento pertenece a Pedido, no a la UI.



Cómo corregirlo:

Mover calcularDescuento() a la clase Pedido. La UI solo llama pedido.calcularDescuento().

Ejercicio 2: La clase Venta hace todo



Escenario:

La clase Venta tiene métodos: procesarPago(), enviarEmailConfirmacion(), guardarEnBaseDeDatos(), generarFactura(), calcularImpuesto() y actualizarInventario().



Pista:

¿Cuántas responsabilidades distintas tiene Venta? ¿Puede describirse en una oración sin 'y'?



Respuesta:

High Cohesion violada — Venta mezcla lógica de negocio, persistencia, email, facturación e inventario. Es una God Class.



Cómo corregirlo:

Separar en: Venta (solo lógica de dominio), VentaRepositorio, NotificadorVenta, GeneradorFactura, ServicioInventario.

Ejercicio 3: El switch de métodos de pago



Escenario:

Cada vez que la empresa añade un nuevo método de pago (crypto, OXXO, transferencia), hay que modificar el método `procesarPago()` de `Caja` que tiene un switch con todos los tipos.



Pista:

¿Qué pasaría si cada tipo de pago implementara su propia lógica? ¿Caja necesitaría conocer los detalles?



Respuesta:

Polymorphism — sin polimorfismo el switch crece con cada nuevo tipo, violando Open/Closed.



Cómo corregirlo:

Crear interfaz `Pago` con método `procesar()`. Cada tipo (`PagoVisa`, `PagoCripto`, `PagoOXXO`) la implementa. `Caja` llama `pago.procesar()` sin if.

Ejercicio 4: Inventario guarda en la BD



Escenario:

La clase Inventario necesita guardar sus datos. El equipo propone añadir un método guardar() directamente en Inventario que ejecute SQL.



Pista:

Si Inventario tiene lógica de negocio Y acceso a base de datos, ¿qué patrón necesita para mantener cohesión?



Respuesta:

Pure Fabrication — mezclar lógica de dominio con persistencia reduce cohesión de Inventario y aumenta acoplamiento a la BD.



Cómo corregirlo:

Crear InventarioRepositorio (Pure Fabrication) con guardar(), buscar(), actualizar(). Inventario queda limpio con solo lógica de negocio.

Ejercicio 5: ServicioReportes crea sus objetos



Escenario:

La clase ServicioReportes usa datos propios para crear objetos de tipo Grafico, Tabla y Exportador directamente con new dentro de sus métodos.



Pista:

ServicioReportes ya TIENE los datos necesarios para crear esos objetos. ¿Qué criterio de Creator se cumple?



Respuesta:

Creator (correcto) — ServicioReportes usa directamente esos objetos y tiene los datos de inicialización. Es el Creator natural.



Cómo corregirlo:

No hay problema. ServicioReportes es el Creator correcto. Pero si esos objetos crecen, considerar Pure Fabrication como fábrica.

Ejercicio 6: PedidoController con 600 líneas



Escenario:

PedidoController tiene 600 líneas. Valida formulario, calcula precios, consulta inventario, aplica descuentos, guarda en BD y envía notificaciones.



Pista:

Un Controller debe COORDINAR, no EJECUTAR. ¿Qué patrones se violan aquí?



Respuesta:

Controller inflado + High Cohesion violada — el Controller ejecuta lógica en lugar de delegarla. Es una God Class disfrazada.



Cómo corregirlo:

Controller solo llama a: ServicioPrecio, InventarioServicio, DescuentoCalculador, PedidoRepositorio, NotificadorPedido.

Lo que aprendiste hoy:



GRASP son criterios de diseño, no código que instalas



Low Coupling + High Cohesion son el objetivo siempre



Information Expert es el más usado del día a día



Controller separa la UI de la lógica de negocio



Polymorphism elimina los if/switch que no paran de crecer



Pure Fabrication mantiene limpias las clases del dominio

*"El código bien diseñado no es aquel que funciona hoy,
sino el que sigue siendo entendible y modificable mañana."*